UAV Software Engineering: ROS 2 Architecture

Advanced Pub/Sub, IPC, and Real-Time Determinism

1 The `rclcpp::Node` Abstraction and DDS Mapping

In ROS 2, every functional component inherits from `rclcpp::Node`. Under the hood, when you call `create_publisher`, the `rclcpp` C++ library translates this request to the `rmw` (ROS Middleware) interface, which maps it to a DDS **DataWriter**. Conversely, `create_subscription` maps to a DDS **DataReader**.

2 Zero-Copy and Intra-Process Communication (IPC)

A naive implementation publishes messages by value (`publish(msg)`), forcing the DDS middleware to serialize the object into a byte stream (CDR) and send it over the loopback network. For high-bandwidth sensors (4K Cameras, LiDAR) residing in the same physical process, ROS 2 provides **Intra-Process Communication (IPC)**. By instantiating a message as a `std::unique_ptr` and publishing it via `std::move()`, the middleware bypasses the network stack and transfers pointer ownership. This results in **Zero-Copy** message passing, eliminating CPU overhead and maintaining strict real-time latency.

3 The Executor Model and Callback Groups

The **Executor** is the engine that spins the node and orchestrates callbacks. To prevent slow callbacks (e.g., logging) from starving high-frequency control loops, ROS 2 uses **Callback Groups**:

1. **Mutually Exclusive (`MutuallyExclusive`):** Callbacks cannot run in parallel. Crucial for protecting shared memory without aggressive `std::mutex` locking.
2. **Reentrant (`Reentrant`):** The executor can spawn multiple threads to execute the exact same callback concurrently if data arrives at high rates.

4 Wall Timers

Writing a blocking `while(true)` loop inside a node permanently blocks the Executor's thread. Background tasks must strictly use **Wall Timers** (`rclcpp::TimerBase`), allowing the Executor to fire the logic deterministically.

5 Real-Time Memory: Custom Allocators (Advanced)

In standard C++, instantiating a message with `new` or `make_unique` uses the OS default allocator. This allocator is non-deterministic: finding a free block of memory in a fragmented heap takes a variable amount of time. In a 1000Hz UAV control loop, a sudden delay of 2ms due to a Page Fault causes a crash. ROS 2 is designed to support **Custom Allocators** (like TLSF - Two-Level Segregated Fit). A true real-time publisher pre-allocates a fixed pool of memory at startup. When publishing, it uses this custom allocator (passed via `rclcpp::PublisherOptions`) to guarantee $O(1)$ constant-time memory allocation, entirely bypassing the OS kernel during flight.

6 Bypassing the Executor: WaitSets (Advanced)

The ROS 2 Executor model carries intrinsic overhead due to mutexes, futures, and thread scheduling. For ultra-low latency flight controllers, relying on the Executor is often unacceptable. ROS 2 exposes the lower-level `rclcpp::WaitSet` API. Instead of letting the Executor invoke callbacks asynchronously, a WaitSet allows the main thread to synchronously block and wait for a specific set of Subscriptions or Timers to trigger. This completely eliminates context-switching and Executor overhead, providing bare-metal deterministic polling—the gold standard for hardware-in-the-loop (HITL) aviation software.